OOP using JAVA

Introduction to Java Programming:

Basics of Java

What is Java?

Java is a **programming language** and a **platform**. Java is a high-level, robust, object-oriented and secure programming language.

Java was developed by *Sun Microsystems* (which is now a subsidiary of Oracle) in the year 1995. *James Gosling* is known as the father of Java. Before Java, its name was *Oak*. Since Oak was already a registered company, so James Gosling and his team changed the name from Oak to Java.

Platform: Any hardware or software environment in which a program runs is known as a platform. Since Java has a runtime environment (JRE) and API, it is called a platform.

Java Example

Let's have a quick look at the Java programming example. A detailed description of the Hello World! example

```
1. public class Main{
2.     public static void main(String args[]){
3.      System.out.println("Hello, World!");
4.     }
5. }
```

Output : Hello World!

Getting Started

Before coding, we need to set up your development environment. Java development typically requires the Java Development Kit (JDK), which includes the Java compiler and other essential tools. User can download the JDK from the official Oracle website and follow the installation instructions for operating system.

Once user have the JDK installed, they can use a text editor or an Integrated Development Environment (IDE) like IntelliJ IDEA, Eclipse, or NetBeans to write and run Java code. IDEs provide features such as code completion,

debugging, and project management, making them invaluable tools for developers.

Application

According to Sun Microsystems, 3 billion devices run Java. There are various devices where Java is currently used. Some of them are as follows:

1. Desktop Applications such as Acrobat Reader, media player, antivirus, etc.
2. Web Applications such as irctc.co.in, tpointtech.com, etc.
3. Enterprise Applications such as banking applications.
4. Mobile
5. Embedded System
6. Smart Card
7. Robotics
8. Games, etc.

Types of Java Applications

There are the following 4-types of applications that can be created using Java programming:

1) Standalone Application

Standalone applications are also known as desktop applications or window-based applications. These are traditional software that we need to install on every machine. Examples of standalone applications are Media players, antivirus, etc. AWT and Swing are used in Java for creating standalone applications.

2) Web Application

An application that runs on the server side and creates a dynamic page is called a web application. Currently, Servlet, JSP, Struts, Spring, Hibernate, JSF, etc. technologies are used for creating web applications in Java.

3) Enterprise Application

An application that is distributed in nature, such as banking applications, etc. is called an enterprise application. It has advantages like high-level security, load balancing, and clustering. In Java, EJB is used for creating enterprise applications.

4) Mobile Application

An application that is created for mobile devices is called a mobile application. Currently, Android and Java ME are used for creating mobile applications.

Java Platforms / Editions

There are four platforms or editions of Java:

1) Java SE (Java Standard Edition)

It is a Java programming platform. It includes Java programming APIs such as java.lang, java.io, java.net, java.util, java.sql, java.math etc. It includes core topics like OOPs, String, Regex, Exception, Inner classes, Multithreading, I/O Stream, Networking, AWT, Swing, Reflection, Collection, etc.

2) Java EE (Java Enterprise Edition)

It is an enterprise platform that is mainly used to develop web and enterprise applications. It is built on top of the Java SE platform. It includes topics like Servlet, JSP, Web Services, EJB, JPA, etc.

3) Java ME (Java Micro Edition)

It is a micro platform that is dedicated to mobile applications.

4) JavaFX

It is used to develop rich Internet applications. It uses a lightweight user interface API.

Background/History of Java

The history of Java is indeed fascinating. Originally designed for interactive television, Java's journey began with the Green Team, a group within Sun Microsystems led by James Gosling. Their goal was to create a programming language for digital devices like set-top boxes and televisions. However, Java's capabilities surpassed the needs of the digital cable television industry at the time. Instead, it found its niche in internet programming, offering a solution that was ahead of its time.

Java's principles, including simplicity, robustness, portability, platform independence, security, high performance, multithreading, architecture neutrality, object orientation, interpretation, and dynamism, laid the foundation for its development. These principles ensured that Java was not only versatile but also adaptable to a wide range of applications.

James Gosling, often referred to as the father of Java, spearheaded the project in the early 1990s. Alongside his team, known as the Green Team, Gosling worked to refine and develop Java into the language we know today. Their efforts culminated in the release of Java in 1995. Netscape later incorporated Java technology into its browser, further propelling its popularity and solidifying its position as a key player in the world of programming languages. From its humble beginnings in digital television to its widespread use on the internet and beyond, Java has continued to evolve, remaining true to its core principles while adapting to the changing needs of the technological landscape.

Currently, Java is used in internet programming, mobile devices, games, e-business solutions, etc. Following are given significant points that describe the history of Java.

1) **James Gosling, Mike Sheridan**, and **Patrick Naughton** initiated the Java language project in June 1991. The small team of sun engineers called **Green Team**.

2) Initially it was designed for small, embedded systems in electronic appliances like set-top boxes.

3) Firstly, it was called "**Greentalk**" by James Gosling, and the file extension was .gt.

4) After that, it was called **Oak** and was developed as a part of the Green project.

Why Java was named as "Oak"?

5) **Why Oak?** Oak is a symbol of strength and chosen as a national tree of many countries like the U.S.A., France, Germany, Romania, etc.

6) In 1995, Oak was renamed as "**Java**" because it was already a trademark by Oak Technologies.

Why Java Programming named "Java"?

7) Why had they chose the name Java for Java language? The team gathered to choose a new name. The suggested words were "dynamic", "revolutionary", "Silk", "jolt", "DNA", etc. They wanted something that reflected the essence of the technology: revolutionary, dynamic, lively, cool, unique, and easy to spell, and fun to say.

According to James Gosling, "Java was one of the top choices along with **Silk**". Since Java was so unique, most of the team members preferred Java than other names.

8) Java is an island in Indonesia where the first coffee was produced (called Java coffee). It is a kind of espresso bean. Java name was chosen by James Gosling while having a cup of coffee nearby his office.

9) Note that Java is just a name, not an acronym.

10) Initially developed by James Gosling at Sun Microsystems (which is now a subsidiary of Oracle Corporation) and released in 1995.

11) In 1995, Time magazine called **Java one of the Ten Best Products of 1995**.

12) JDK 1.0 was released on January 23, 1996. After the first release of Java, there have been many additional features added to the language. Now Java is being used in Windows applications, Web applications, enterprise applications, mobile applications, cards, etc. Each new version adds new features in Java.

Java Version History

The latest stable release is **Java 22 (2024)**.

- **JDK Alpha & Beta (1995):** Early test versions showcasing Java's platform independence.
- **JDK 1.0 (1996):** First official release with JVM, Applets, and core libraries.
- **JDK 1.1 (1997):** Added JDBC, RMI, JavaBeans, and AWT event model.
- **J2SE 1.2 (1998):** Introduced Swing, Collections Framework, and JNDI.
- **J2SE 1.3 (2000):** Added HotSpot JVM, JNDI, and Sound API.
- **J2SE 1.4 (2002):** Introduced assert, NIO, regex, and XML support.
- **J2SE 5.0 (2004):** Added generics, annotations, enums, and enhanced for-loop.
- **Java SE 6 (2006):** Introduced scripting and Java Compiler API.
- **Java SE 7 (2011):** Added try-with-resources, diamond operator, and fork/join.
- **Java SE 8 (2014):** Major update with lambdas, Streams, and java.time API.
- **Java SE 9 (2017):** Introduced modular system (JPMS) and JShell.
- **Java SE 10 (2018):** Added var for local variable type inference.
- **Java SE 11 (2018):** LTS release; added HttpClient API and removed old APIs.

- **Java SE 12–16 (2019–2021):** Introduced switch expressions, text blocks, records, sealed classes, and pattern matching.
- **Java SE 17 (2021):** LTS release with sealed classes and Foreign Function API.
- **Java SE 18–20 (2022–2023):** Improved pattern matching, virtual threads, and scoped values.
- **Java SE 21 (2023):** LTS release with String Templates and Sequenced Collections.

Java and the Internet

Java is an object-oriented, class-based programming language. The language is designed to have as few dependencies implementations as possible. The intention of using this language is to give relief to the developers from writing codes for every platform. The term WORA, write once and run everywhere is often associated with this language. It means whenever we compile a Java code, we get the byte code (.class file), and that can be executed (without compiling it again) on different platforms provided they support Java. In the year 1995, Java language was developed. It is mainly used to develop web, desktop, and mobile devices. The Java language is known for its robustness, security, and simplicity features. That is designed to have as few implementation dependencies as possible.

Features of Java

The primary objective of Java programming language creation was to make it portable, simple and secure programming language. Apart from this, there are also some excellent features which play an important role in the popularity of this language. The features of Java are also known as Java buzzwords.

1. Simple
2. Object-Oriented
3. Portable
4. Platform independent
5. Secured
6. Robust
7. Architecture neutral
8. Interpreted
9. High Performance
10. Multithreaded

11. Distributed
12. Dynamic

Simple

Java is very easy to learn, and its syntax is simple, clean and easy to understand. According to Sun Microsystem, Java language is a simple programming language because:

- Java syntax is based on C++ (so easier for programmers to learn it after C++).
- Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.
- There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

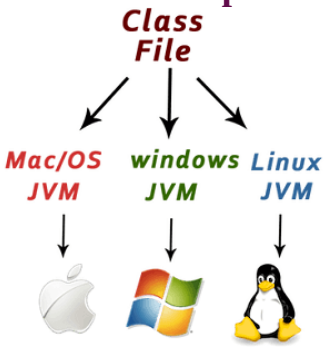
Object-oriented

Java is an object-oriented programming language. Everything in Java is an object. Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behavior.

Basic concepts of OOPs are:

1. Object&Class
2. Inheritance
3. Polymorphism
4. Abstraction
5. Encapsulation

Platform Independent



Java is platform independent because it is different from other languages like C, C++, etc. which are compiled into platform specific machines while Java is a write once, run anywhere language. A platform is the hardware or software environment in which a program runs.

There are two types of platforms software-based and hardware-based. Java provides a software-based platform.

It has two components:

1. Runtime Environment
2. API(Application Programming Interface)

Secured

Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:

- **No explicit pointer**
- **Java Programs run inside a virtual machine sandbox**

Robust

The English mining of Robust is strong. Java is robust because:

- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- Java provides automatic garbage collection which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- There are exception handling and the type checking mechanism in Java. All these points make Java robust.

Architecture-neutral

Java is architecture neutral because there are no implementation dependent features, for example, the size of primitive types is fixed.

In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.

Portable

Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.

High-performance

Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code. It is still a little bit slower than a compiled language (e.g., C++). Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

Distributed

Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.

Multi-threaded

A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc.

Dynamic

Java is a dynamic language. It supports the dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.

Advantages of Java

- **Platform Independent** – Write once, run anywhere using the JVM.
- **Object-Oriented** – Supports modular, reusable, and maintainable code.
- **Secure** – Built-in security features like bytecode verification and no pointers.
- **Robust** – Strong memory management and exception handling.
- **Multithreaded** – Supports concurrent execution of multiple tasks.
- **Portable** – Runs on any system without modification.
- **High Performance** – JIT compiler improves execution speed.
- **Distributed** – Supports networking and remote method invocation (RMI).
- **Dynamic** – Supports dynamic loading of classes at runtime.
- **Rich API** – Provides extensive libraries for development.

Java Virtual Machine & Byte Code

Java Virtual Machine (JVM)

- **Definition:**
  The Java Virtual Machine (JVM) is a **software-based engine** that runs Java programs. It acts as a bridge between the **compiled Java code (bytecode)** and the **underlying operating system**.
- **Function:**
  When you run a Java program, the **JVM converts bytecode into machine code** that your computer can understand and execute.
- **Key Features:**
  1. **Platform Independence:**
     JVM allows Java code to run on any device that has a compatible JVM installed — this is what makes Java "Write Once, Run Anywhere."
  2. **Memory Management:**
     JVM automatically manages memory using **Garbage Collection**, freeing unused objects.
  3. **Security:**
     It verifies bytecode before execution to prevent malicious code from running.
  4. **Performance:**
     The **Just-In-Time (JIT) Compiler** improves performance by converting frequently used bytecode into native code.
- **Main Components of JVM:**
  - **Class Loader:** Loads .class files into memory.
  - **Bytecode Verifier:** Checks code for errors and security issues.
  - **Interpreter / JIT Compiler:** Executes bytecode line by line or compiles it into native code.
  - **Runtime Data Areas:** Includes **Heap**, **Stack**, **Method Area**, and **Program Counter** for managing data during execution.

Java Bytecode

- **Definition:**
  Bytecode is an **intermediate, platform-independent code** generated by the Java compiler (javac). It is stored in .class files.
- **Role:**
  The **JVM reads and executes bytecode**, translating it into machine-specific instructions at runtime.

- **Why Bytecode?**
  - Makes Java **platform-independent**.
  - Ensures **security** by being verified before execution.
  - Improves **portability** — the same .class file can run on Windows, Linux, or macOS.
- **Example Flow:**

  Source Code → (javac) → Bytecode (.class) → (JVM) → Machine Code

- **Advantages:**
  - Faster execution with JIT compilation
  - Portable across platforms
  - Compact and efficient

Java Environment Setup

To write and run Java programs, you need to **install and configure the Java Development Environment** on your computer.

1. Components of Java Environment

The Java programming environment mainly consists of three parts:

1. **JDK (Java Development Kit):**
   - It contains tools needed to **develop Java programs**.
   - Includes **compiler (javac)**, **JRE**, and **development tools** like debugger and jar.
   - Used by programmers for **writing, compiling, and testing** Java code.
2. **JRE (Java Runtime Environment):**
   - It is required to **run** Java programs.
   - Contains the **JVM**, core classes, and supporting libraries.
   - End users only need JRE to execute Java applications.
3. **JVM (Java Virtual Machine):**
   - The engine that **executes Java bytecode**.
   - Converts bytecode into machine code for the system.

2. Steps to Set Up Java

Step 1: Download JDK

- Go to Oracle's Java website or OpenJDK.
- Download the correct version for your OS (Windows, macOS, or Linux).

Step 2: Install JDK

- Run the installer and follow the setup wizard instructions.
- By default, it installs in:
  C:\Program Files\Java\jdk-<version>\

Step 3: Set Environment Variables

To use Java commands from the command prompt:

1. Open **System Properties → Environment Variables**
2. Add a new variable:
   - **Variable name:** JAVA_HOME
   - **Variable value:** C:\Program Files\Java\jdk-<version>
3. Add %JAVA_HOME%\bin to the **Path** variable.

Step 4: Verify Installation

Open **Command Prompt** and type:

```
java -version
javac -version
```

If both show version numbers, Java is successfully installed. ✅

3. Writing and Running Java Program

Write Code: Save file as Hello.java

```
class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

Compile Code:

javac Hello.java

- This creates Hello.class (bytecode file)

Run Program:

java Hello

Output: Hello, Java!

Java Program Structure

A Java program is made up of **classes, methods, and statements** that work together to perform specific tasks.
Every Java program must have at least **one class** and a **main() method** where execution begins.

Basic Structure of a Java Program

```
// Example: Simple Java Program
// This is a comment

// Package Declaration (optional)
package mypackage;

// Import Statement (optional)
import java.util.*;

// Class Declaration
public class HelloWorld {

    // main() method - starting point of every Java program
    public static void main(String[] args) {

        // Statement - prints text to the screen
        System.out.println("Hello, World!");
    }
```

Parts of a Java Program

1. **Package Declaration (optional)**
    - Defines the package (folder) in which the class belongs.
    - Example: package mypackage;
2. **Import Statements (optional)**
    - Used to include built-in or user-defined classes from other packages.
    - Example: import java.util.Scanner;
3. **Class Declaration**

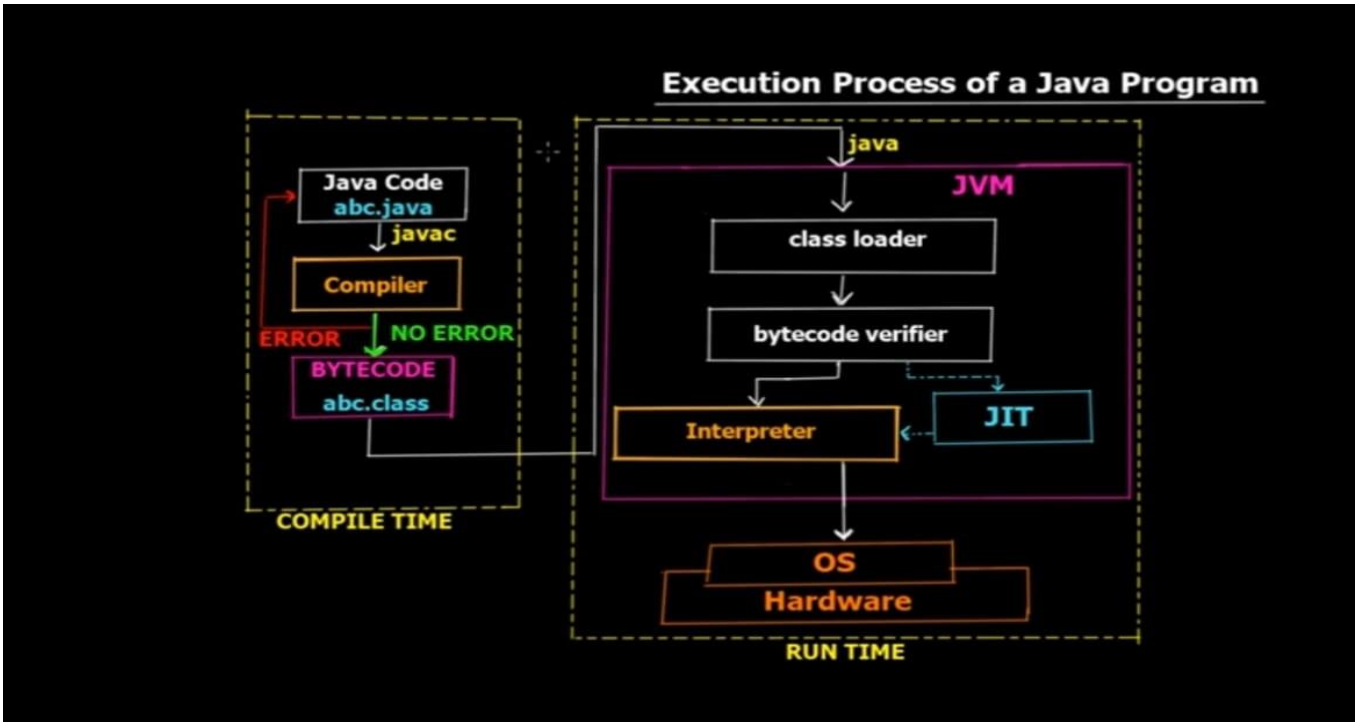
- Every Java program must have at least one class.
- The class name should match the file name.
- Example: public class HelloWorld { ... }

4. **main() Method**
   - The entry point of the program where execution starts.
   - Syntax:
   - public static void main(String[] args)
5. **Statements**
   - Instructions inside the main method that perform actions.
   - Example: System.out.println("Hello, World!");
6. **Comments**
   - Used to explain the code. They are ignored by the compiler.
   - Single-line: // comment
   - Multi-line: /* comment */

Points to Consider :

- File name and public class name **must be the same** (e.g., HelloWorld.java).
- Execution starts from the **main() method**.
- Curly braces {} define the **scope** of classes and methods.
- **Semicolon (;)** marks the end of each statement.

How Java Program is executes?



Basics of OOP:

OOPs Concepts in Java

Object-oriented programming is a paradigm that provides concepts, such as **inheritance, data binding, polymorphism**, etc.

Simula is considered the first object-oriented programming language. The programming paradigm where everything is represented as an object is known as a pure object-oriented programming language.

Smalltalk is considered the first truly object-oriented programming language.

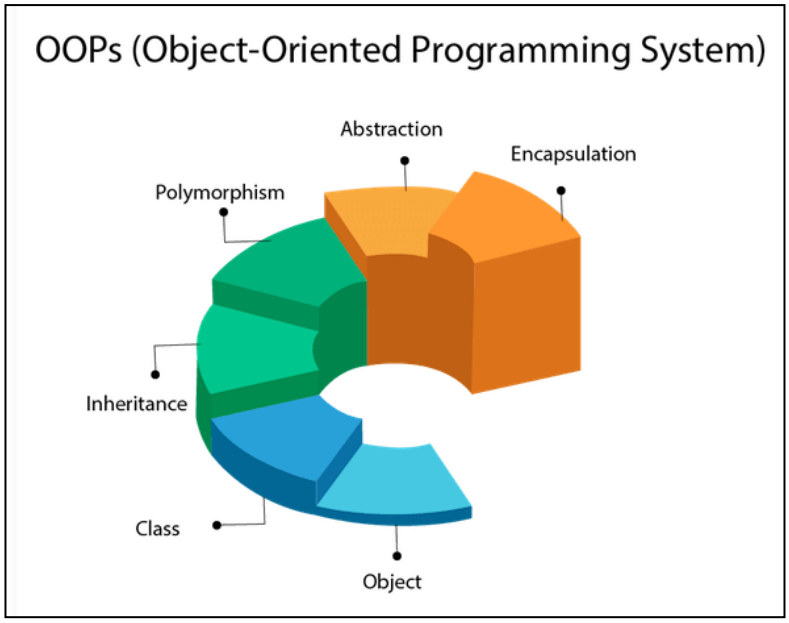
The popular object-oriented languages are Java, C#, PHP, Python, C++, etc.

OOPs (Object-Oriented Programming)

Object-oriented programming aims to implement real-world entities, for example, objects, classes, abstraction, inheritance, polymorphism, etc.

Object means a real-world entity such as a pen, chair, table, computer, watch, etc. **Object-oriented programming** is a methodology or paradigm for designing a program using classes and objects. It simplifies software development and maintenance by providing some concepts:

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation



Object

Any entity that has state and behavior is known as an object. For example, a chair, pen, table, keyboard, bike, etc. It can be physical or logical.

An Object can be defined as an instance of a class. It contains an address and takes up some space in memory. Objects can communicate without knowing the details of each other's data or code. The only necessary thing is the type of message accepted and the type of response returned by the objects.

Example: A dog is an object because it has states like color, name, breed, etc. as well as behaviors like wagging the tail, barking, eating, etc.

The primary elements of an object are:

- **State:** It is embodied in an object's properties. Additionally, it reflects an object's attributes.
- **Behavior:** An object's methods are a representation of it. Additionally, it represents how an object reacts to other objects.
- **Identity:** Identity is a special name that gives a thing the ability to communicate with other objects.
- **Method:** A method is a group of statements that carry out a certain operation and give an outcome. A method can complete a certain task without giving back any results. Methods are regarded as time savers since they enable us to reuse the code without having to type it again.

Java differs from languages like C, C++, and Python in that each method needs to be a component of a class.

Example

```
1. class Dog {
2.     String name;
3.     void bark() {
4.         System.out.println(name + " says Woof!");
5.     }
6. }
7. public class Main {
8.     public static void main(String[] args) {
9.         Dog myDog = new Dog();     // Creating an object
10.        myDog.name = "Rocky";      // Assigning value to the object's field
11.        myDog.bark();              // Calling method on the object
12.    }
13.}
```

Output:

Rocky says Woof!

Class

Class is a logical entity. A class can also be defined as a blueprint from which we can create an individual object. Class does not consume any space.

In general, a class declaration includes the following in the order as it appears:

1. **Modifiers:** A class can be public or have default access.
2. **class keyword:** The class keyword is used to create a class.
3. **Class name:** The name must begin with an initial letter (capitalized by convention).
4. **Superclass (if any):** The name of the class's parent (superclass), if any, preceded by the keyword extends. A class can only extend (subclass) one parent.
5. **Interfaces (if any):** A comma-separated list of interfaces implemented by the class, if any, preceded by the keyword implements. A class can implement more than one interface.
6. **Body:** The class body surrounded by braces, { }.

Inheritance

When one object acquires all the properties and behaviors of a parent object, it is known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.

Example

```
1. // Superclass (Parent)
2. class Vehicle {
3.     void start() {
4.         System.out.println("Vehicle is starting...");
5.     }
6.     void stop() {
7.         System.out.println("Vehicle is stopping...");
8.     }
9. }
10.// Subclass (Child) - Inherits from Vehicle
11.class Car extends Vehicle {
12.    void honk() {
13.        System.out.println("Car is honking!");
14.    }
15.}
16.public class MainExample {
17.    public static void main(String[] args) {
18.        Car myCar = new Car();
19.        // Calling inherited methods (from Vehicle)
20.        myCar.start();
21.        myCar.stop();
22.        // Calling child class method
23.        myCar.honk();
24.    }
25.}
```

Output:

Vehicle is starting...
Vehicle is stopping...
Car is honking!

Polymorphism

If one task is performed in different ways, it is known as polymorphism. For example, to convince the customer differently, to draw something, for example, a shape, a triangle, a rectangle, etc.

In Java, we use method overloading and method overriding to achieve polymorphism.

Another example can be to speak something; for example, a cat says meow, a dog barks woof, etc.

Example

```
1. class Animal {
2.     // Method Overloading (compile-time polymorphism)
3.     void sound() {
4.         System.out.println("An animal makes a sound");
5.     }
6.     void sound(String type) {
7.         System.out.println("Animal sound: " + type);
8.     }
9. }
10.class Dog extends Animal {
11.    // Method Overriding (runtime polymorphism)
12.    @Override
13.    void sound(String type) {
14.        System.out.println("Dog barking is: " + type);
15.    }
16.}
17.public class Main {
18.    public static void main(String[] args) {
19.        Animal a = new Animal();
20.        Dog d = new Dog();
21.        Animal poly = new Dog();
22.        // Method Overloading
23.        a.sound();
24.        a.sound("Generic");
25.        // Method Overriding
26.        d.sound("Loud");
27.        // Performing the Polymorphism
28.        poly.sound("Soft");
29.    }
30.}
```

Output:

An animal makes a sound
Animal sound: Generic
Dog barking is: Loud
Dog barking is: Soft

Abstraction

Hiding internal implementation and showing functionality only to the user is known as abstraction. For example, in phone calls, we do not know the internal processing. In Java, we use abstract classes and interfaces to achieve abstraction.

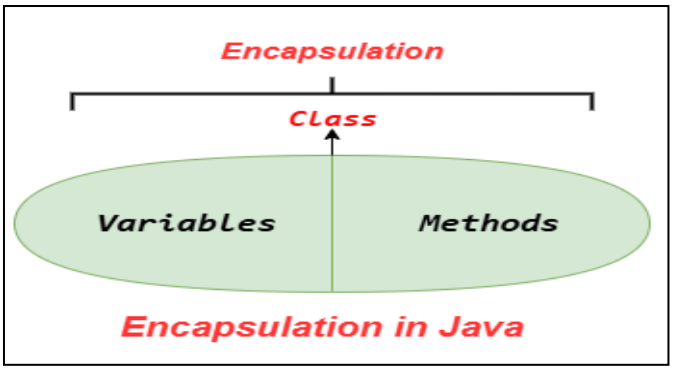
Example of Abstraction

```
1. abstract class Animal {
2.     abstract void makeSound();
3.     // Creating a Concrete method
4.     void breathe() {
5.         System.out.println("Animal is breathing...");
6.     }
7. }
8. // Concrete subclass
9. class Dog extends Animal {
10.    // implementation for an abstract method
11.    @Override
12.    void makeSound() {
13.        System.out.println("Dog barking");
14.    }
15.}
16.public class Main {
17.    public static void main(String[] args) {
18.        Animal myDog = new Dog();
19.        myDog.breathe();
20.        myDog.makeSound();
21.    }
22.}
```

Output:

Animal is breathing...
Dog barking

Encapsulation



Binding (or wrapping) code and data together into a single unit is known as encapsulation. For example, a capsule is wrapped with different medicines.

A Java class is an example of encapsulation. A Java bean is a fully encapsulated class because all the data members are private.

Example of Abstraction

Example

```
1. class Student {
2.     // Private data members
3.     private String name;
4.     // Setter method
5.     public void setName(String name) {
6.         this.name = name;
7.     }
8.     // Getter method
9.     public String getName() {
10.        return name;
11.    }
12.}
13.public class Main {
14.    public static void main(String[] args) {
15.        Student s = new Student();
16.        // Setting value using setter
17.        s.setName("John");
18.        // Getting value using getter
19.        System.out.println("Student Name: " + s.getName());
20.    }
```

21.}

Output:

Student Name: John

Subclasses and super classes

1. Superclass (Parent Class)

- A **superclass** is the class whose **properties (fields) and behaviors (methods)** are inherited by another class.
- Also called the **parent class** or **base class**.
- Declared normally with the class keyword.

Example:

```
class Animal {        // Superclass
    void eat() {
        System.out.println("Animal is eating");
    }
}
```

2. Subclass (Child Class)

- A **subclass** is the class that **inherits from a superclass**.
- Also called the **child class** or **derived class**.
- Uses the keyword extends to inherit from a superclass.
- Can **add new features** or **override superclass methods**.

Example:

```
class Dog extends Animal {    // Subclass
    void bark() {
        System.out.println("Dog is barking");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();   // inherited from Animal
```

```
        d.bark();  // subclass method
    }
}
```

Output:

Animal is eating
Dog is barking

Key Points

1. **Inheritance:** Subclass inherits all **non-private** members of superclass.
2. **Method Overriding:** Subclass can provide its own implementation of a superclass method.
3. **Single Inheritance in Java:** A class can extend **only one superclass**.
4. **"is-a" Relationship:**
    - Subclass **is a** type of superclass.
    - Example: Dog **is an** Animal.

Polymorphism and Overloading

1. Polymorphism

Definition:
Polymorphism means **"one name, many forms"**. In Java, it allows a single entity (method or object) to behave differently in different contexts.

Types of Polymorphism in Java:

1. **Compile-time Polymorphism (Static Polymorphism)**
    - Achieved using **method overloading** or **operator overloading** (Java does not support custom operator overloading).
    - Method name is same but **parameters are different**.
2. **Runtime Polymorphism (Dynamic Polymorphism)**
    - Achieved using **method overriding**.
    - Method in subclass **overrides** a method in superclass.
    - Decided at runtime which method to call.

Example (Runtime Polymorphism):

```
class Animal {
    void sound() {
```

```
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();  // Runtime polymorphism
        a.sound();             // Output: Dog barks
    }
}
```

2. Method Overloading

Definition:
Method overloading is **defining multiple methods with the same name in the same class** but with **different parameters** (number, type, or order).

Rules for Overloading:

1. Same method name
2. Different parameter list
3. Can have different return type (optional)

Example:

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();
```

```
        System.out.println(c.add(5, 10));      // Output: 15
        System.out.println(c.add(5.5, 3.2));   // Output: 8.7
    }
}
```

Key Points

Feature	Method Overloading	Runtime Polymorphism
Also Called	Compile-time polymorphism	Dynamic polymorphism
When Resolved	At compile time	At runtime
Parameters	Must be different	Can be same
Return Type	Can be different	Must be compatible
Example	add(int a, int b) vs add(double a, double b)	Dog overrides Animal's sound()

Dynamic Binding

Binding is a mechanism creating link between method call and method actual implementation. As per the polymorphism concept in Java, object can have many different forms. Object forms can be resolved at compile time and run time.

Java Dynamic Binding

Dynamic binding refers to the process in which linking between method call and method implementation is resolved at run time (or, a process of calling an overridden method at run time). Dynamic binding is also known as run-time polymorphism or late binding. Dynamic binding uses objects to resolve binding.

Characteristics of Java Dynamic Binding

- **Linking** − Linking between method call and method implementation is resolved at run time.
- **Resolve mechanism** − Dynamic binding uses object type to resolve binding.
- **Example** − Method overriding is the example of Dynamic binding.
- **Type of Methods** − Virtual methods use dynamic binding.

Java Dynamic Binding: Using the super Keyword

When invoking a superclass version of an overridden method the **super** keyword is used so that we can utilize parent class method while using dynamic binding.

Method overriding example

Program:

```
Animals can move
Dogs can walk and run
```

```
class Animal {
public void move() {
System.out.println("Animals can move");
} }
class Dog extends Animal {
public void move() {
System.out.println("Dogs can walk and run");
} }
public class Tester {
public static void main(String args[]) {
Animal a = new Animal(); // Animal reference and object
// Dynamic Binding
Animal b = new Dog(); // Animal reference but Dog object
a.move(); // runs the method in Animal class
b.move(); // runs the method in Dog class
} }
```

Example: Using Super Keyword

Program:

```
class Animal {
public void move() {
System.out.println("Animals can move");
} }
class Dog extends Animal {
public void move() {
super.move(); // invokes the super class method
System.out.println("Dogs can walk and run");
} } public class TestDog {
public static void main(String args[]) {
Animal b = new Dog(); // Animal reference but Dog
object
b.move(); // runs the method in Dog class
} }
```

```
Animals can move
Dogs can walk and run
```

Message communication

Message communication in Java refers to the way **objects interact and exchange information** by **sending messages (method calls) to each other**.

In **object-oriented programming**, objects do not directly access each other's data. Instead, they **request actions** from other objects via **methods** — this is called **message passing**.

How Message Communication Works

1. **Sender Object:**
   - The object that wants some action to be performed.
   - Sends a message by **calling a method** on another object.
2. **Receiver Object:**
   - The object that receives the message and executes the method.

- Performs the requested action using its own data or methods.

3. **Method Call:**
   - The **message** in Java is essentially a **method invocation**.
   - Example: object.methodName(parameters);

Example

```
class Printer {
   void printMessage(String msg) {
      System.out.println(msg);
   }
}

class Main {
   public static void main(String[] args) {
      Printer p = new Printer();      // Receiver object
      p.printMessage("Hello Java!");  // Sender sends message via method call
   }
}
```

Output:

Hello Java!

Key Points

- Message communication ensures **encapsulation** — objects communicate without exposing internal data.
- Promotes **modularity** and **reusability**.
- Forms the **core of object-oriented programming**: objects interact by passing messages instead of sharing data directly.

Procedure-Oriented vs. Object-Oriented Programming concept.

Procedure Oriented Programming

Definition : Procedural Programming is a programming language that follows a step-by-step approach to break down a task into a collection of variables and routines (or subroutines) through a sequence of instructions. Each step is carried out in order in a systematic manner so that a computer can understand what to do.
Approach : In procedural programming, the main program is divided into small parts based on the functions and is treated as separate program for individual smaller program.
Access modifiers : No such modifiers are introduced in procedural programming.
Security : Procedural programming is less secure as compare to OOPs.
Complexity : There is no simple process to add data in procedural programming, at least not without revising the whole program.
Program division : Procedural programming divides a program into small programs and each small program is referred to as a function.
Importance : Procedural programming does not give importance to data. In POP, functions along with sequence of actions are followed.
Inheritance : Procedural programming does not provide any inheritance.
Examples : C, BASIC, COBOL, Pascal, etc. are the examples POP languages.

Object Oriented Programming

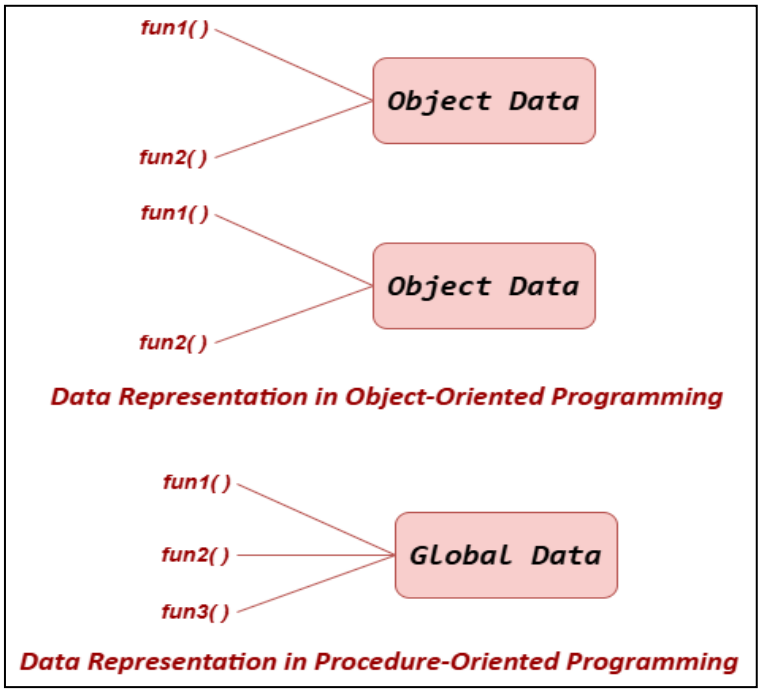
Definition : Object-oriented Programming is a programming language that uses classes and objects to create models based on the real world environment. In OOPs, it makes it easy to maintain and modify existing code as new objects are created inheriting characteristics from existing ones.
Approach : In OOPs concept of objects and classes is introduced and hence the program is divided into small chunks called objects which are instances of classes.
Access modifiers : In OOPs access modifiers are introduced namely as Private, Public, and Protected.
Security : Due to abstraction in OOPs data hiding is possible and hence it is more secure than POP.
Complexity : OOPs due to modularity in its programs is less complex and hence new data objects can be created easily from existing objects making object-oriented programs easy to modify
Program division : OOP divides a program into small parts and these parts are referred to as objects.
Importance : OOP gives importance to data rather than functions or procedures.
Inheritance : OOP provides inheritance in three modes i.e. protected, private, and public
Examples : C++, C#, Java, Python, etc. are the examples of OOP languages.

Advantages of OOP Over Procedure-Oriented Programming Language

1. OOPs make development and maintenance easier, whereas in a procedure-oriented programming language, it is not easy to manage if the code grows as the project size increases.
2. OOP provides data hiding, whereas in a procedure-oriented programming language, global data can be accessed from anywhere.
3. OOPs provide the ability to simulate real-world events much more effectively. We can provide the solution to real-world problems if we use an Object-Oriented Programming language.

In short:

- **OOP languages** are *fully object-oriented* (support all principles).
- **Object-based languages** are *partially object-oriented* — they use objects but lack features like inheritance and polymorphism.



What is the difference between an object-oriented programming language and an object-based programming language?

Feature	Object-Oriented Programming (OOP)	Object-Based Programming
Definition	Languages that support all the features of Object-Oriented Programming like inheritance, polymorphism, encapsulation, and abstraction.	Languages that support the use of objects but do **not** support all OOP features, especially **inheritance**.
Inheritance	Supported	**Not supported**
Polymorphism & Abstraction	Supported	**Not supported**
Example Languages	Java, C++, Python, C#	JavaScript (before ES6), VBScript
Reusability	High — due to inheritance and polymorphism	Limited — no inheritance
Encapsulation	Supported	Supported (through objects)

Java compilation and execution process

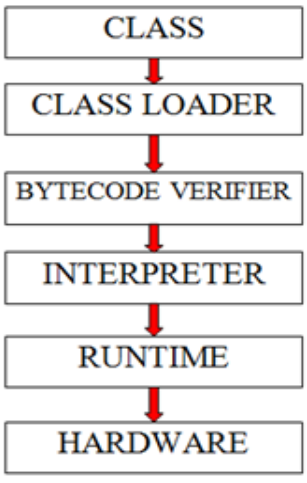
Compile time

Converts java code to byte code using java compiler

Ex : abc.java file is converted into abc.class file

This class file is used for execution

Runtime :



JVM

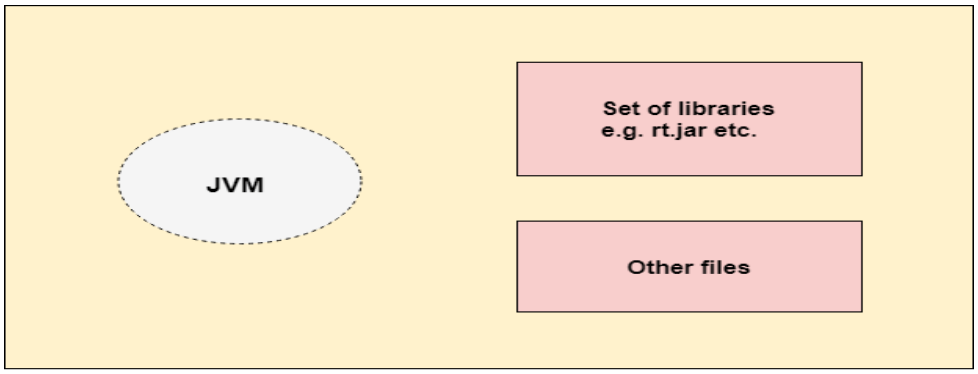
JVM (Java Virtual Machine) is an abstract machine. It is called a virtual machine because it doesn't physically exist. It is a specification that provides a runtime environment in which Java bytecode can be executed. It can also run those programs which are written in other languages and compiled to Java bytecode.

JVMs are available for many hardware and software platforms. JVM, JRE, and JDK are platform dependent because the configuration of each OS is different from each other. However, Java is platform independent. There are three notions of the JVM: *specification*, *implementation*, and *instance*.

The JVM performs the following main tasks:

- Loads code
- Verifies code
- Executes code
- Provides runtime environment

JRE

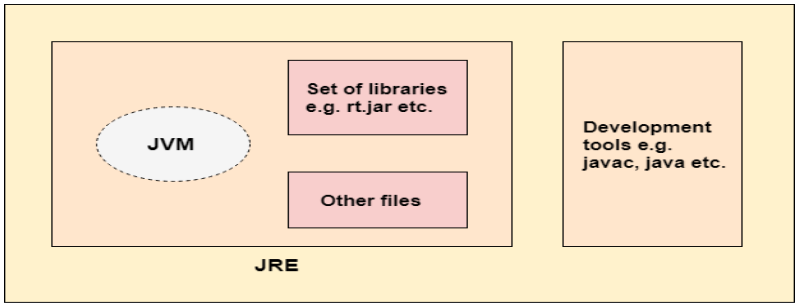


JRE

JRE is an acronym for Java Runtime Environment. It is also written as Java RTE. The Java Runtime Environment is a set of software tools which are used for developing Java applications. It is used to provide the runtime environment. It is the implementation of JVM. It physically exists. It contains a set of libraries + other files that JVM uses at runtime.

The implementation of JVM is also actively released by other companies besides Sun Micro Systems.

JDK



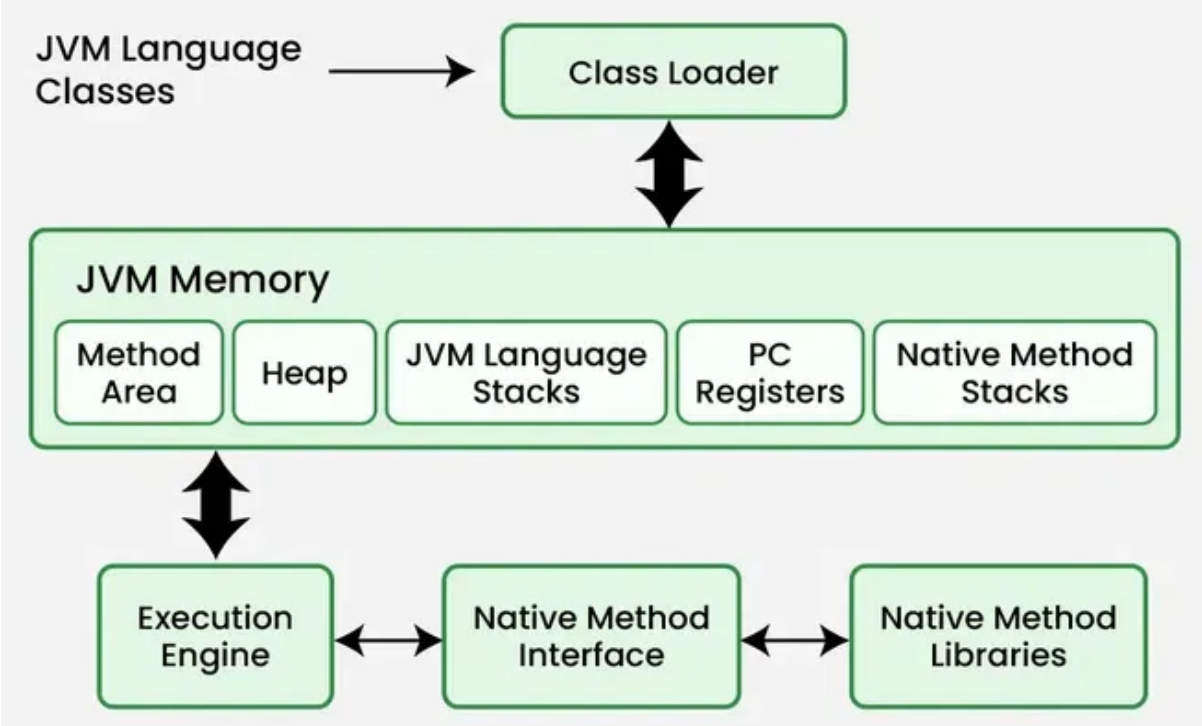
JDK is an acronym for Java Development Kit. The Java Development Kit (JDK) is a software development environment which is used to develop Java applications and applets. It physically exists. It contains JRE + development tools.

JDK is an implementation of any one of the below given Java Platforms released by Oracle Corporation:

- Standard Edition Java Platform
- Enterprise Edition Java Platform
- Micro Edition Java Platform

- The JDK contains a private Java Virtual Machine (JVM) and a few other resources such as an interpreter/loader (java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc), etc. to complete the development of a Java Application.

Java Virtual Machine Architecture



The **JVM** is the **runtime engine** of Java that executes compiled bytecode and provides platform independence. Its architecture consists of several components that work together to load, verify, and execute Java programs.

1. Class Loader Subsystem

- **Purpose:** Loads class files (compiled .class bytecode) into JVM memory.
- **Key Features:**
  - Loads classes **on demand** (lazy loading).
  - Uses **three types of class loaders**:
    1. **Bootstrap Class Loader** – loads core Java classes (java.lang.*).
    2. **Extension Class Loader** – loads classes from ext directory.
    3. **System / Application Class Loader** – loads user-defined classes from classpath.
- **Tasks Performed:**

1. **Loading:** Reads .class files.
2. **Linking:** Verifies, prepares, and optionally resolves symbols.
3. **Initialization:** Initializes static variables and executes static blocks.

2. Runtime Data Areas

JVM manages different **memory areas** at runtime:

Area	Description
Method Area	Stores class structures: metadata, static variables, constant pool. Shared among all threads.
Heap	Stores all Java objects and arrays. Garbage collected.
Stack	Stores **stack frames** for each thread, including local variables, partial results, and method call info.
PC Register (Program Counter)	Holds address of **current instruction** being executed for each thread.
Native Method Stack	Contains info for executing **native (non-Java) methods**, like C/C++ code.

3. Execution Engine

- **Purpose:** Executes bytecode loaded by the class loader.
- **Components:**
    1. **Interpreter:** Executes bytecode **line by line** (slower but simple).
    2. **Just-In-Time (JIT) Compiler:** Converts bytecode into **native machine code** for faster execution.
    3. **Garbage Collector:** Automatically manages memory by removing unused objects.

4. Native Interface

- **Purpose:** Allows JVM to interact with **native libraries** (e.g., C/C++ DLLs or shared libraries).
- **Example:** Java Native Interface (JNI) lets Java code call native methods.

5. Native Method Libraries

- Libraries required by the JVM for **executing native code** and interacting with the operating system.

6. JVM Workflow Summary

1. **Class Loading:** Classes are loaded into the method area.
2. **Bytecode Verification:** Ensures code integrity and security.
3. **Execution:** Interpreter or JIT executes instructions.

4. **Memory Management:** Stack, heap, and method area are managed; garbage collector frees unused objects.

Note

- JVM is **platform-independent**: write once, run anywhere.
- Handles **memory management, execution, and security**.
- Works with **class loaders, runtime data areas, execution engine, and native interface**.

Difference between JRE, JDK, JVM

Feature	JVM (Java Virtual Machine)	JRE (Java Runtime Environment)	JDK (Java Development Kit)
Definition	Abstract machine that runs Java **bytecode**.	Provides the **runtime environment** to run Java programs.	Complete **Java development kit** for writing, compiling, and running Java programs.
Purpose	Executes Java programs and makes them platform-independent.	Runs Java applications on your system.	Develops and runs Java programs.
Components	Execution engine, runtime data areas, class loader, native interface.	JVM + libraries required to run Java programs.	JRE + tools like javac, jar, javadoc.
Contains Compiler?	No	No	Yes (javac)
Who Uses It?	End-user or program at runtime.	End-user to run programs.	Developers to **write, compile, and run** programs.
Output	Runs .class files (bytecode).	Runs Java programs.	Produces bytecode (.class) from source code (.java).